Web Engineering & Development (SWE 363)

Back-end Development Fundamentals II

In today's lecture:

- Node.js
- npm
- Modules

Reference:

- Zybook: 6.2-6.3

6.2 Node.js

What is Node.js?

> **Node.js** is a JavaScript runtime environment, mainly used to run **server-side** web applications

- **Event-driven, non-blocking I/O** → handles high loads
- **JavaScript** on both the server and the client
- Simple way to create and share **modules**
- Works well with **MongoDB** and **JSON**

Used by: Netflix, Walmart, eBay, LinkedIn

Running Node.js

Interactive shell

```
$ node
> console.log("Hello, Node.js!")
Hello, Node.js!
undefined
> x = 2
2
> .exit
```

Run a file

```
// hello.js
for (let i = 0; i < 3; i++) {
  console.log("Hello, Node.js!");
}
```

```
$ node hello.js
```

Activity: Predict the output

1. What does the shell display?

```
$ node
> console.log("Hi")
> x = 5
> .exit
```

2. True or False: `node test.js` starts the interactive shell.

3. True or False: Node.js programs run in the web browser.

A Simple Web Server: the `http` Module

```
// server.js
const http = require("http");

http.createServer(function(request, response) {
  response.writeHead(200, {"Content-Type": "text/html"});
  response.write("<h1>Hello, Node.js!</h1>");
  response.end();
}).listen(3000);
```

- `require()` → imports a module (CommonJS)
- `createServer()` → creates the server; the callback runs on each HTTP request
- `writeHead()` → sets the **status code** and content type
- `write()` / `end()` → writes the HTML / **sends** the response
- `listen(3000)` → starts listening → `http://localhost:3000/` (stop with **Ctrl+C**)

npm & Node.js Projects

- **Project** → a directory of JavaScript files, packages, and configuration files
- **Package** → a directory with one or more modules + a `package.json` file
- **npm** (Node Package Manager) → downloads, installs, and updates packages
  - Installed with Node.js: `npm -v`

```
myproject
├── node_modules
│   └── underscore
├── package.json
├── package-lock.json
└── server.js
```

Local vs. Global Install

Local (default)

```
npm install underscore
```

- Saved in the project's `node_modules`
- For **project dependencies**

Global (`-g`)

```
npm install nodemon --global
```

- Saved in `{prefix}/node_modules`
- For **command-line tools**

Common npm Commands

Command	Example
Install a package locally / globally	`npm install lodash / npm install nodemon -g`
Install **all** project dependencies	`npm install`
List local / global packages	`npm list / npm list -g`
Update a package	`npm update lodash`
Uninstall a package	`npm uninstall lodash`
Show the global install location	`npm config get prefix`

package.json

```
{
  "name": "my-web-server",
  "version": "1.0.0",
  "description": "A simple web server",
  "main": "server.js",
  "dependencies": { "underscore": "^1.13.6" },
  "scripts": { "start": "node server.js" },
  "license": "ISC"
}
```

- Lists the project's name, version, license, and **dependencies**
- Create it by hand or with `npm init`
- `npm install` → installs **all** dependencies listed
- `"start"` script → `npm start` runs `node server.js`

Versions & package-lock.json

Semantic versioning: `major.minor.patch` → `1.9.1`

- **major** → new functionality; may change how old functions work
- **minor** → bug fixes and minor changes
- **patch** → a bug fix to a minor version

`"^1.9.1"` → highest version with major number **1** (1.9.2 yes, 2.0.0 no)

package-lock.json → records the **exact** versions to use

- The same versions on every machine
- Created or modified when dependencies are added or removed

Activity: Answer the following questions

1. In the `http` server, which method starts listening for requests: `require()`, `createServer()`, or `listen()`?
2. What URL accesses a web server running locally on port 8080?
3. Where does `npm install grunt -g` install the package: `./node_modules` or `{prefix}/node_modules`?
4. `"lodash": "^4.17.0"` (^ keeps the major number), and versions `4.17.21`, `4.18.0`, `5.0.0` exist. With no package-lock.json, which is installed?
5. A project has package.json but no `node_modules`. Which command installs everything: `npm install`, `npm list`, or `npm update`?

Using a Third-Party Package

```
npm install underscore
```

```
const _ = require("underscore");   // underscore is usually named _

// Use underscore to get a random number between 1 and 6
let randNum = _.random(1, 6);
```

- The module is stored in `node_modules/underscore`
- Good practice: name the variable after the module (`http` for "http")

Activity: Predict

```
const http = require("http");
const _ = require("underscore");

http.createServer(function(request, response) {
  response.writeHead(200, {"Content-Type": "text/html"});
  for (let i = 0; i < 5; i++) {
    let randNum = _.random(1, 6);
    response.write("<p>" + randNum + "</p>");
  }
  response.end();
}).listen(3000);
```

1. How many numbers does `http://localhost:3000/` show? In what range?
2. What happens when you refresh the page?
3. What happens if `npm install underscore` was never run?

6.3 Modules

Why Modules?

Separate files

```
// flights.js
const max = 10;
function purchaseTicket() { ... }

// hotels.js
const max = 10;   // Error: max conflicts
function bookRoom() { ... }
```

```
<script src="flights.js"></script>
<script src="hotels.js"></script>
```

Separate modules

```
// flights.js
const max = 10;
function purchaseTicket() { ... }
export { purchaseTicket };

// hotels.js
const max = 10;   // no error
function bookRoom() { ... }
export { bookRoom };
```

You control **what is exported**

Two Module Formats

ES Modules

- Supported by all major browsers
- `import` / `export`

```
import { bookRoom } from "./hotels.js";
```

CommonJS

- Commonly used in Node.js
- `require()`

```
const http = require("http");
```

Exporting

Named exports (many)

```
// hello.js
function sayHello() { alert("Hello!"); }
function sayHi() { alert("Hi!"); }

export { sayHello, sayHi };
```

or before each declaration:

```
export function sayHello() { ... }
```

Default export (one)

```
// goodbye.js
function sayGoodbye() { alert("Goodbye!"); }

export default sayGoodbye;
```

or before the declaration:

```
export default function sayGoodbye() { ... }
```

Importing

```
import { sayHello, sayHi } from "./hello.js";   // non-default → braces
import sayGoodbye from "./goodbye.js";          // default → no braces
```

- `"./"` → the file is in the **same directory**

In a webpage:

```
<script type="module">
  import { sayHello, sayHi } from "./hello.js";
  sayHello();
</script>
```

- `type="module"` is **required** for `import` to work
- Files must be on a **web server** (not opened with `file://`)

Activity: What happens if (each change on its own)

```
// hello.js
export { sayHello, sayHi };

// goodbye.js
export default sayGoodbye;
```

```
// index.html
<script type="module">
  import { sayHello, sayHi } from "./hello.js";
  import sayGoodbye from "./goodbye.js";
  sayHello(); sayHi(); sayGoodbye();
</script>
```

1. The call `sayHi();` is removed from index.html?
2. `sayHi` is removed from the export in hello.js?
3. `type="module"` is removed from the `<script>` tag?
4. The import uses `"hello.js"` instead of `"./hello.js"`?
5. `export default function sayAdios() {...}` is added to goodbye.js?

Activity: Find the 3 bugs

```
// hello.js
function sayHello() { alert("Hello!"); }

// goodbye.js
export default function sayGoodbye() { alert("Goodbye!"); }
export default function sayAdios() { alert("Adios!"); }
```

```
<script type="module">
  import { sayHello } from "./hello.js";
  import { sayGoodbye } from "./goodbye.js";
  sayHello();
  sayGoodbye();
</script>
```

Key Takeaways

- **Node.js** runs JavaScript on the server: `node file.js`
- The **http** module: `createServer()`, `writeHead()`, `end()`, `listen()`
- **npm** installs packages locally (`node_modules`) or globally (`-g`)
- **package.json** = dependency ranges; **package-lock.json** = exact versions
- **Modules:** named exports `{ }` or one default export; local paths start with `./`

30m

Demo

6.2 CLI calculator

Build a CLI calculator using:

- npm packages
- Custom modules

Next Class

- Express.js
- Building web servers